

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



Crop Disease Knowledge Assistant

A Retrieval-Augmented Generation (RAG) System

Project ID: T044

Team ID: PRJ044

Team GitHub Repository: https://github.com/praveenreddykana-maker/crop-disease-knowledge-.git

Student Name	Registration Number	Individual GitHub Link
Kana Praveen Kumar Reddy	99230041019	https://github.com/praveenreddykana-maker/GEN_AI-mastary.git
Konda Naveen Reddy	99230041024	https://github.com/naveenreddykonda13-ctrl/GRN-AI-MASTARY.git
N. Pedda Peeraiah	99230040988	https://github.com/peeraiahnadendla-glitch/peeraiahnadendla78-GEN-MASTERY
Vennapusa Madhu Sudhan Reddy	99230041007	https://github.com/vennapusamadhusudhanreddy
P. Gurunath Reddy	99230040953	https://github.com/GURUREDDY18/GEN-Mastary

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1

2. Problem Statement & Objectives ..... Section 2

3. System Architecture (RAG Pipeline) ..... Section 3

4. Implementation Details & Tech Stack ..... Section 4

5. Results and Testing ..... Section 5

6. Conclusion & Future Scope ..... Section 6

1. Introduction

Crop diseases cause serious yield losses for farmers, and timely, correct information is often hard to find. Reliable guidance is scattered across lengthy research papers, government publications, and agricultural guidelines. General-purpose Large Language Models (LLMs) can answer farming questions, but they may hallucinate, for example by inventing pesticide names, dosages, or references. In agriculture this can lead to real harm.

Retrieval-Augmented Generation (RAG) addresses this problem by retrieving relevant passages from a trusted knowledge base before the LLM generates an answer. The Crop Disease Knowledge Assistant is a web application that ingests agricultural documents and answers questions about crop diseases, covering symptoms, causes, prevention, and management. Every answer is grounded in retrieved documents and shown with its sources.

2. Problem Statement & Objectives

Problem: Farmers, students, and agricultural extension workers spend excessive time searching through long PDF documents to find information on a specific crop disease. Keyword search does not capture meaning, so a query such as "yellow spots on tomato leaves that curl upward" may not match the document's wording. Generic chatbots, on the other hand, can give unsupported or unsafe advice.

Objectives:

- To build an AI assistant that answers crop disease questions based strictly on uploaded, trusted agricultural documents.
- To implement a vector database for semantic search and efficient retrieval of relevant context.
- To reduce hallucinations by forcing the LLM to answer only from retrieved chunks and to cite them.
- To make the assistant say clearly when sufficient information is not available.
- To provide an administrator interface for managing the knowledge base.

- To deliver a simple, responsive, agricultural-themed interface that non-technical users can use.

3. System Architecture (RAG Pipeline)

The architecture is divided into two phases: Data Ingestion and Query Processing.

3.1 Data Ingestion Phase

An administrator uploads an agricultural document (PDF or similar). The system then:

1. Validates the uploaded file (type, size).
2. Extracts text using PyPDF.
3. Cleans the text (removes headers, footers, extra whitespace, and broken lines).
4. Splits the text into overlapping chunks.
5. Converts each chunk into an embedding using a Sentence Transformers model.
6. Stores the vectors, along with metadata (document name, page number, crop), in the vector database (FAISS or ChromaDB).

The processing status of each document (uploaded, processing, completed, failed) is tracked and shown in the admin panel.

3.2 Query & Generation Phase

1. The user submits a question through the chat interface.
2. The question is converted into an embedding using the same model used at ingestion.
3. The vector database performs a semantic similarity search and returns the top-K most relevant chunks. K and the similarity threshold are configurable.
4. The retrieved chunks are inserted into the prompt as context, together with the question.
5. The LLM generates an answer using only that context.
6. The answer is returned in a fixed format with source citations (document name and page).
7. If no relevant chunks are found, the assistant states that it does not have enough information.

3.3 Structured Response Format

Every answer follows this structure:

1. Disease Name
2. Affected Crop
3. Symptoms
4. Possible Causes
5. Prevention Methods

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend	React.js, Tailwind CSS	Responsive dashboard and chat interface
Backend	Python, FastAPI	REST API, file handling, chat logic
Orchestration Framework	LangChain	Connecting loaders, retrievers, vector store, and LLM
Document Processing	PyPDF	Text extraction from PDFs
Embeddings Model	Sentence Transformers (e.g., all-MiniLM-L6-v2)	Converting text into vector representations
Vector Database	FAISS or ChromaDB	Storing and retrieving document embeddings
Large Language Model	Gemini API (or another supported LLM)	Generating answers from retrieved context
Configuration	Environment variables (.env)	Securing API keys and settings

---

4.1 Sample Chunking Configuration

To preserve context across chunk boundaries, a Recursive Character Text Splitter with overlap is used:

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len
)
```

5. Results and Testing

The app has seven main parts, described here in simple words.

- Dashboard: the home screen. It shows the app's name and logo, a welcome message, the chatbot, your recent questions, the crops covered, and a menu on the side.
- Chatbot: you type a question such as "What causes tomato leaf curl?" and get an answer, as in a modern AI assistant. The answer comes only from the uploaded documents.
- Crop disease library: diseases are organized by crop (Rice, Wheat, Tomato, Potato, Cotton, Maize, Groundnut). Each disease shows its name, symptoms, causes, favorable conditions, prevention, management, and sources.
- Chat history: old conversations are saved. You can search them, delete one, or clear everything.
- Admin panel: a control area only administrators can use. They upload or remove documents and check whether each document has finished processing.
- Security and reliability: only valid files can be uploaded, only admins manage documents, secret keys are kept out of the code, errors show clear messages, and every answer shows real sources.
- Design: a green and yellow farm theme that works on phones and computers, with a side menu, icons, cards, and loading indicators while the AI thinks.

---

6. Conclusion & Future Scope

Conclusion: The Crop Disease Knowledge Assistant combines semantic search over a vector database with the generative capabilities of an LLM to deliver accurate, source-backed information about crop diseases. By restricting answers to retrieved context and requiring citations, the system reduces hallucination and supports safer decision-making for farmers and agricultural learners.

Future Scope:

- Image-based disease identification, allowing users to upload a photo of a diseased leaf.
- Multilingual support, including regional languages such as Telugu, Tamil, and Hindi.
- Voice input and output for farmers with limited literacy.
- User authentication with role-based access.

- A feedback mechanism to evaluate and improve retrieval accuracy.
- Integration of weather data to warn about conditions favorable to specific diseases.
- Offline or low-bandwidth support for rural areas.